Projet PageRank
Sujet 8:Le Backspace pour les pages sans lien de sortie

Comparaison de l'approche Google et de l'approche Backspace

OUEDRAOGO Abibou SAMBIENI Flora

Mai 2026

Contents

1 Introduction et présentation du problème **3**
1.1 PageRank et le surfeur aléatoire . . . 3
1.2 Le problème des dangling nodes . . . 3
1.3 Deux approches pour traiter les dangling nodes . . . 3

2 Formalisation mathématique **3**
2.1 Notations . . . 3
2.2 Construction du graphe Backspace . . . 4
2.3 Agrégation des scores . . . 4
2.4 Métriques de comparaison . . . 4

3 Implémentation en C **5**
3.1 Organisation du code . . . 5
3.2 L'itération de PageRank . . . 5
3.3 La transformation Backspace . . . 6
3.4 Tri et métriques . . . 6

4 Données utilisées **6**

5 Expériences et résultats **7**
5.1 Partie 1 : variation du paramètre α . . . 7
5.2 Partie 2 : impact du nombre de dangling nodes . . . 9
5.3 Limite de calcul au-delà de 40 % de destruction . . . 9
5.4 Courbes de convergence . . . 10

6 Comparaison sur trois graphes : California, EPA et HEP-th **11**
6.1 Présentation des trois graphes . . . 11
6.2 Comportement de ρ et τ en fonction de α . . . 11
6.3 Impact de la destruction de liens . . . 12
6.4 Comportement universel de la convergence . . . 12

7 Analyse et discussion **13**
7.1 Complexité computationnelle 13
7.2 Pertinence du modèle Backspace 13
7.3 Cas du parent lui-même dangling 14

8 Conclusion **14**

1 Introduction et présentation du problème

1.1 PageRank et le surfeur aléatoire

L'algorithme PageRank modélise la navigation d'un surfeur aléatoire : à chaque étape, avec probabilité α il suit un lien sortant de la page courante, et avec probabilité $1-\alpha$ il se téléporte vers une page quelconque selon la distribution z. La distribution stationnaire de ce processus de Markov définit le score de chaque page.

Soit H la matrice stochastique par colonnes ($H_{ji} = 1/d^+(i)$ si $i \to j$), l'itération s'écrit :

$$x^{(k+1)} = \alpha \left(H\, x^{(k)} + \sum_{i \in D} x_i^{(k)} \right) z + (1-\alpha)\, z \tag{1}$$

où $D = \{i : d^+(i) = 0\}$ est l'ensemble des dangling nodes et $z_i = 1/N$ par défaut.

1.2 Le problème des dangling nodes

Un **dangling node** est une page de degré sortant nul : le surfeur y est piégé et la masse de probabilité accumulée n'est jamais redistribuée par les liens. Deux situations se distinguent :

- $d^-(P) > 0$: la page a des prédécesseurs. C'est le cas traité dans ce projet.
- $d^-(P) = 0$: la page est totalement isolée. Aucun utilisateur ne peut y arriver par navigation ; la question du Backspace ne se pose pas. Les deux approches traitent ce cas de façon identique via la téléportation.

1.3 Deux approches pour traiter les dangling nodes

Approche Google. La masse d'un dangling node P est redistribuée uniformément vers toutes les pages, comme si P possédait un lien vers chacune. Cette approximation perd l'information sur le chemin de navigation.

Approche Backspace (ce projet). Un utilisateur arrivant sur une page sans lien appuie sur Backspace et revient à la page précédente. Pour conserver l'information d'où l'on vient, on duplique chaque dangling node P (avec $d^-(P) > 0$) en autant d'exemplaires qu'il a de prédécesseurs.

Objectif : comparer les rankings obtenus par ces deux approches en faisant varier $\alpha \in [0.05, 0.95]$, et mesurer l'impact du nombre de dangling nodes.

2 Formalisation mathématique

2.1 Notations

- N : nombre de pages dans le graphe original.
- $d^+(i)$, $d^-(i)$: degrés sortant et entrant de la page i.
- $D = \{i : d^+(i) = 0\}$: ensemble des dangling nodes.

- H : matrice stochastique par colonnes du graphe original.
- z : vecteur de téléportation ($z_i = 1/N$, distribution initiale uniforme).
- $x^{(k)}$: vecteur de scores à l'itération k.
- x : distribution stationnaire (vecteur PageRank final).
- $\alpha \in (0, 1)$: facteur d'amortissement (paramètre principal).
- N_{trans} : taille du graphe étendu après transformation Backspace.

2.2 Construction du graphe Backspace

Soit $P \in D$ avec $\text{pred}(P) = \{X_1, \ldots, X_{D^-}\}$, $D^- = d^-(P) > 0$. La transformation se fait en quatre étapes :

1. Transférer les pages normales ($d^+(i) > 0$) dans le graphe étendu via un tableau de correspondance `correspondance[i]`.
2. Pour chaque prédécesseur $X_k \in \text{pred}(P)$: créer le clone (P, X_k), remplacer $X_k \to P$ par $X_k \to (P, X_k)$, ajouter $(P, X_k) \to X_k$ (Backspace).
3. Recopier tous les arcs entre pages normales.
4. Recalculer les degrés et listes de voisins entrants.

Le graphe étendu a $N_{\text{trans}} = |\{i : d^+(i) > 0\}| + \sum_{P \in D} d^-(P)$ nœuds et ne contient plus de dangling nodes.

2.3 Agrégation des scores

Après convergence sur le graphe étendu, on ramène les scores dans l'espace original :

$$\tilde{x}_i = x_i^{\text{ext}} + \sum_{\substack{c \text{ clone} \\ \text{origine}(c)=i}} x_c^{\text{ext}}, \quad \text{puis} \quad \tilde{x} \leftarrow \frac{\tilde{x}}{\|\tilde{x}\|_1} \tag{2}$$

2.4 Métriques de comparaison

Corrélation de Spearman ρ. Notons $d_p = \text{rang}_G(p) - \text{rang}_B(p)$ la différence de rang :

$$\rho = 1 - \frac{6 \sum_{p=0}^{N-1} d_p^2}{N(N^2 - 1)} \in [-1, 1] \tag{3}$$

$\rho = 1$: rankings identiques ; $\rho = -1$: rankings inversés.

Distance de Kendall τ.

$$\tau = \frac{\#\{\text{paires } (i, j) \text{ discordantes}\}}{\binom{N}{2}} \in [0, 1] \tag{4}$$

$\tau = 0$: identiques ; $\tau = 1$: totalement inversés. Ces deux métriques sont complémentaires : ρ est sensible aux grandes différences de rang, τ compte directement les inversions de paires.

3 Implémentation en C

3.1 Organisation du code

Le programme est structuré en huit blocs fonctionnels :

1. `lire_graphe` : formats Matrix Market (.mtx, base 1) et texte (base 0), avec vérification de dépassement de `MAX_VOISINS` à chaque écriture.
2. `sauvegarder_graphe` / `restaurer_graphe` : appelées en début de chaque partie pour garantir l'indépendance des expériences.
3. `detruire_liens_sortants` : transforme une fraction aléatoire de pages normales en dangling nodes.
4. `transformer_backspace` : construit le graphe étendu (4 étapes).
5. `pagerank_sur` : algorithme générique pour les deux approches ; enregistre la courbe de convergence dans `historique_diff[]`.
6. `agreger_scores_backspace` : agrégation des scores des clones.
7. `calculer_spearman`, `calculer_kendall` : s'appuient sur `calculer_rangs` qui utilise `qsort` en $O(N \log N)$.
8. Exports CSV : `resultats_alpha.csv`, `resultats_taux.csv`, `convergence_google_085.csv`, `convergence_backspace_085.csv`.

3.2 L'itération de PageRank

```
// Correction dangling : masse redistribuee uniformement
double correction = 0.0;
for (int j = 0; j < n; j++)
    if (dp[j] == 0) correction += r[j] / n;

// Mise a jour de chaque page i
for (int i = 0; i < n; i++) {
    r_new[i]  = (1.0 - alpha) / n;   // teleportation
    r_new[i] += alpha * correction;   // dangling
    for (int k = 0; k < ve[i].taille; k++) {
        int j = ve[i].liste[k];
        r_new[i] += alpha * r[j] / dp[j]; // liens entrants
    }
}
// Norme L1 pour la courbe de convergence
double diff = 0.0;
for (int i = 0; i < n; i++) diff += fabs(r_new[i] - r[i]);
historique_diff[iter] = diff;
```

La même fonction est appelée sur le graphe original (approche Google, taille N) et sur le graphe étendu (approche Backspace, taille N_{trans}).

3.3 La transformation Backspace

Algorithm 1 Transformation Backspace (`transformer_backspace`)

```
correspondance[i] ← −1 pour tout i                    ▷ Init. à −1 : détecte les accès invalides
N_trans ← 0
for i = 0 to N − 1 do                                  ▷ Étape 1 : pages normales
    if d⁺(i) > 0 then
        correspondance[i] ← N_trans; origine[N_trans] ← i; N_trans++
    end if
end for
for P = 0 to N − 1 do                                  ▷ Étape 2 : clones dangling
    if d⁺(P) = 0 and d⁻(P) > 0 then
        for X ∈ pred(P) do
            if correspondance[X] = −1 then ignorer     ▷ X est aussi dangling
            end if
            Créer clone c ; origine[c] ← P
            Ajouter arcs X → c et c → X (Backspace)
            N_trans++
        end for
    end if
end for
Recopier arcs normaux → normaux                        ▷ Étape 3
Recalculer d⁺_trans, d⁻_trans, voisin_entrant_trans    ▷ Étape 4
```

Points de robustesse.

- `correspondance[]` initialisé à -1 : si un prédécesseur est lui-même dangling (cas dégénéré), l'accès invalide est détecté et le lien est ignoré avec un avertissement.
- Vérification `taille < MAX_VOISINS` avant chaque écriture dans `liste[]`, protégeant contre les corruptions mémoire silencieuses sur les graphes à hubs très connectés.
- `restaurer_graphe()` appelé explicitement en début des deux parties, garantissant que chaque expérience part du graphe original.

3.4 Tri et métriques

La version initiale utilisait des tris à bulles $O(N^2)$ dupliqués dans plusieurs fonctions. La version finale factorise le tri dans `calculer_rangs()`, qui appelle `qsort` en $O(N \log N)$. Pour $N = 9\,664$, cela représente un facteur d'accélération d'environ $N/\log_2 N \approx 730$.

4 Données utilisées

Les expériences sont menées sur le graphe *Pajek/California* issu de la collection SuiteSparse Matrix Collection [1]. Ce graphe modélise les pages web retournées par une recherche sur le mot-clé *California*.

Création de dangling nodes. Si un graphe ne contient pas de dangling nodes, la fonction `detruire_liens_sortants(taux)` retire tous les liens sortants d'une fraction

Propriété	Valeur
Nombre de pages (N)	9 664
Nombre d'arcs	16 150
Pages sans lien sortant	4 637 (48%)
Pages après transformation Backspace (N_{trans})	13 103

Table 1: Caractéristiques du graphe California

`taux` de pages normales, choisies aléatoirement. Les listes de voisins entrants sont mises à jour en conséquence. Cette procédure est utilisée dans la Partie 2.

5 Expériences et résultats

5.1 Partie 1 : variation du paramètre α

Protocole. On fixe le graphe California avec ses dangling nodes originaux et on fait varier α sur 19 valeurs régulièrement espacées : $\alpha \in \{0.05, 0.10, 0.15, \ldots, 0.90, 0.95\}$. Pour chaque valeur, on calcule PageRank Google et PageRank Backspace, puis ρ (Spearman) et τ (Kendall).

α	ρ (Spearman)	τ (Kendall)	Itér. Google
0.05	0.3804	0.3685	7
0.10	0.4068	0.3604	9
0.15	0.4013	0.3592	11
0.20	0.4019	0.3611	12
0.25	0.4206	0.3497	14
0.30	0.4257	0.3476	16
0.35	0.4078	0.3568	19
0.40	0.4164	0.3522	21
0.45	0.3934	0.3638	24
0.50	0.4167	0.3518	28
0.55	0.4228	0.3489	32
0.60	0.3963	0.3624	38
0.65	0.4271	0.3473	44
0.70	0.4303	0.3457	54
0.75	0.4506	0.3376	66
0.80	0.4370	0.3426	85
0.85	**0.4401**	**0.3414**	**116**
0.90	0.4422	0.3408	178
0.95	0.4029	0.3585	362

Table 2: Corrélation de Spearman ρ, distance de Kendall τ et nombre d'itérations Google selon α (graphe California, 19 valeurs testées). La ligne en gras correspond à la valeur standard $\alpha = 0.85$.

Interprétation.

- $\alpha \to 0$: les deux méthodes convergent vers la distribution de téléportation uniforme

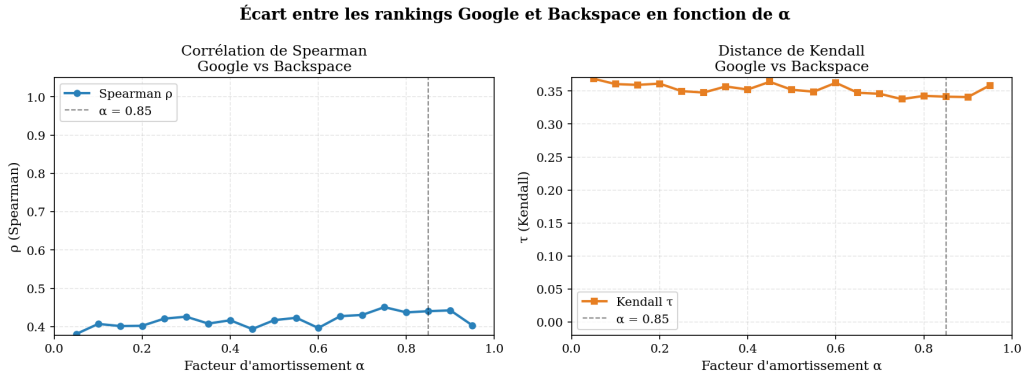


Figure 1: Corrélation de Spearman ρ (gauche) et distance de Kendall τ (droite) entre les rankings Google et Backspace, en fonction de α.

z, indépendamment de la structure du graphe. Les rankings sont quasi-identiques ($\rho \to 1$, $\tau \to 0$).

- $\alpha \to 1$: la structure du graphe domine. Le traitement des dangling nodes a un impact maximal : c'est là que les deux approches divergent le plus.
- $\alpha \in [0.80, 0.90]$: plage pertinente en pratique (Google utilise $\alpha = 0.85$). Les différences sont significatives et illustrent bien l'intérêt de l'approche Backspace. Au-delà de $\alpha = 0.95$, la convergence n'est plus garantie en pratique (1000 itérations insuffisantes sur ce graphe).

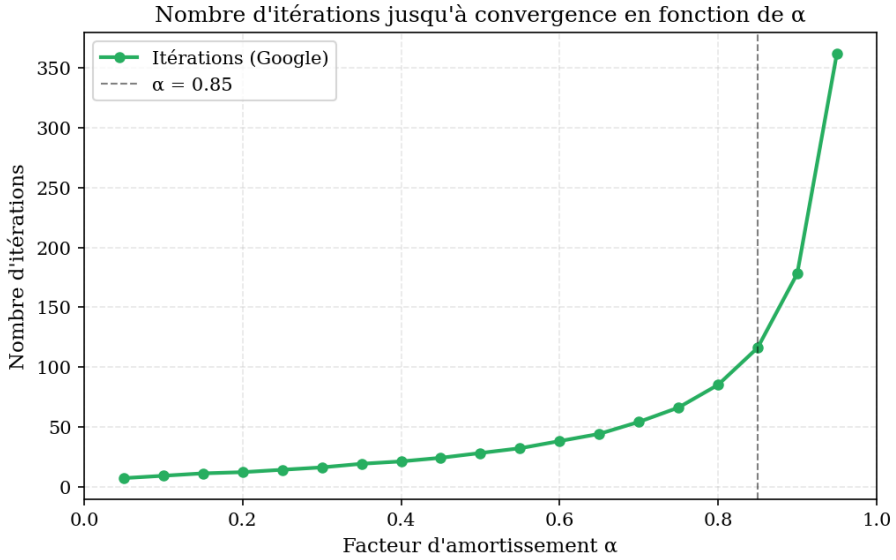


Figure 2: Nombre d'itérations jusqu'à convergence en fonction de α. Plus α est grand, plus le graphe domine sur la téléportation et plus la convergence est lente.

Quelles pages sont affectées ? L'approche Backspace favorise les prédécesseurs des dangling nodes : leur masse de probabilité leur revient intégralement via le Backspace, au lieu d'être dispersée uniformément sur tout le graphe. Les pages qui pointent vers des dangling nodes gagnent des rangs, tandis que les pages globalement populaires (pages 1487 et 4390 dans notre jeu de données) peuvent légèrement perdre du rang.

5.2 Partie 2 : impact du nombre de dangling nodes

Protocole. On fixe $\alpha = 0.85$ et on fait varier le taux de destruction des liens sortants. Le graphe original est restauré avant chaque valeur de taux (`restaurer_graphe`), puis les liens sont détruits aléatoirement (`detruire_liens_sortants`).

Taux destruction	Pages sans sortie	ρ (Spearman)	τ (Kendall)
0%	4 637 (48.0%)	0.4395	0.3402
5%	4 888 (50.6%)	0.4178	0.3490
10%	5 159 (53.4%)	0.4392	0.3338
15%	5 391 (55.8%)	0.4341	0.3397
20%	5 659 (58.5%)	0.4447	0.3325
25%	5 929 (61.3%)	0.4605	0.3230
30%	6 114 (63.3%)	0.4698	0.3203
40%	6 628 (68.6%)	0.4594	0.3216

Table 3: Corrélation de Spearman ρ et distance de Kendall τ selon le taux de destruction de liens ($\alpha = 0.85$, 8 valeurs testées). Au-delà de 40%, le calcul exact de la distance de Kendall devient infaisable sur le graphe California (voir section 5.3).

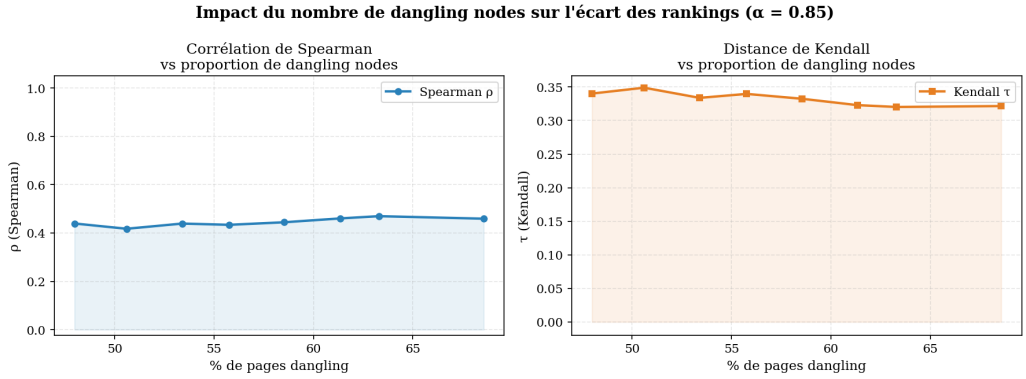


Figure 3: Corrélation de Spearman ρ (gauche) et distance de Kendall τ (droite) en fonction du pourcentage de dangling nodes ($\alpha = 0.85$).

Interprétation. Les résultats montrent une tendance générale : τ diminue (de 0.340 à 0.322) quand le pourcentage de dangling nodes augmente de 48% à 69%, ce qui indique que les rankings se rapprochent paradoxalement. Ce résultat s'explique : quand on détruit massivement des liens, de nombreuses pages deviennent isolées et obtiennent des scores quasi-nuls dans les deux méthodes — elles sont donc classées de façon similaire en bas du ranking par les deux approches. ρ présente des oscillations dues à l'aspect aléatoire de la destruction de liens (la graine aléatoire n'est pas fixée). Quand $|D|/N \to 0$, les deux méthodes coïncideraient ($\tau \to 0$, $\rho \to 1$).

5.3 Limite de calcul au-delà de 40 % de destruction

Observation expérimentale. Pour le graphe California ($N = 9\,664$ pages), le calcul devient impraticable au-delà d'un seuil de destruction d'environ 40 % des liens sortants. À partir de 50 % de liens détruits, le programme ne répond plus lors du calcul du PageRank

sur le graphe transformé par l'approche Backspace, rendant l'expérience impossible à terminer en temps raisonnable.

Explication. Ce blocage a deux origines distinctes :

1. **Explosion de la taille du graphe étendu.** La transformation Backspace duplique chaque dangling node en autant de clones qu'il a de prédécesseurs. Quand le taux de destruction croît, le nombre de clones augmente très rapidement. À 40 % de destruction, $N_{\text{trans}} \approx 9\,853$ nœuds ; au-delà, le graphe étendu peut atteindre plusieurs dizaines de milliers de nœuds, saturant les tableaux statiques alloués à la compilation (`MAX_NOEUDS`, `MAX_VOISINS`).

2. **Instabilité de la convergence PageRank.** Avec une proportion de dangling nodes dépassant 70 %, la matrice de transition du graphe étendu devient très creuse. La masse de probabilité est piégée dans de nombreuses composantes quasi-isolées, ralentissant drastiquement l'itération de puissance, voire la bloquant si la condition d'ergodicité n'est plus vérifiée numériquement.

Conséquence sur les résultats. Les expériences de la Partie 2 sont donc limitées à des taux de destruction $\leq 40\,\%$ ($\leq 69\,\%$ de dangling nodes). Cette plage reste suffisante pour observer la tendance principale (τ décroissant, ρ légèrement croissant) et valider l'interprétation structurelle. Pour aller au-delà, il faudrait soit utiliser des allocations dynamiques (au lieu de tableaux statiques), soit estimer τ par un estimateur Monte-Carlo sur des paires aléatoires plutôt qu'en énumérant les $\binom{N}{2}$ paires exactes.

5.4 Courbes de convergence

Protocole. Pour $\alpha = 0.85$, on enregistre à chaque itération $\|x^{(k)} - x^{(k-1)}\|_1$ pour les deux méthodes.

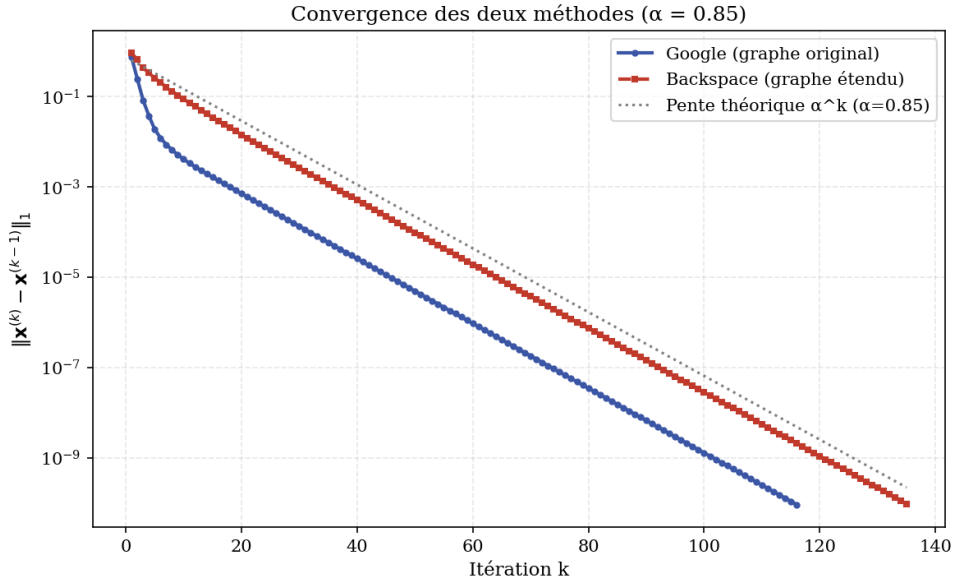


Figure 4: Convergence des deux méthodes pour $\alpha = 0.85$ (échelle semi-logarithmique). La droite pointillée montre la pente théorique α^k.

Interprétation. Les deux courbes sont linéaires en échelle log-linéaire, ce qui confirme la convergence géométrique de l'itération de puissance. Le taux théorique est $\alpha = 0.85$,

soit une pente de $\log_{10}(0.85) \approx -0.071$ par itération. La méthode Backspace opère sur un graphe légèrement plus grand ($N_{\text{trans}} \geq N$) mais converge au même taux α, ce qui est conforme à la théorie.

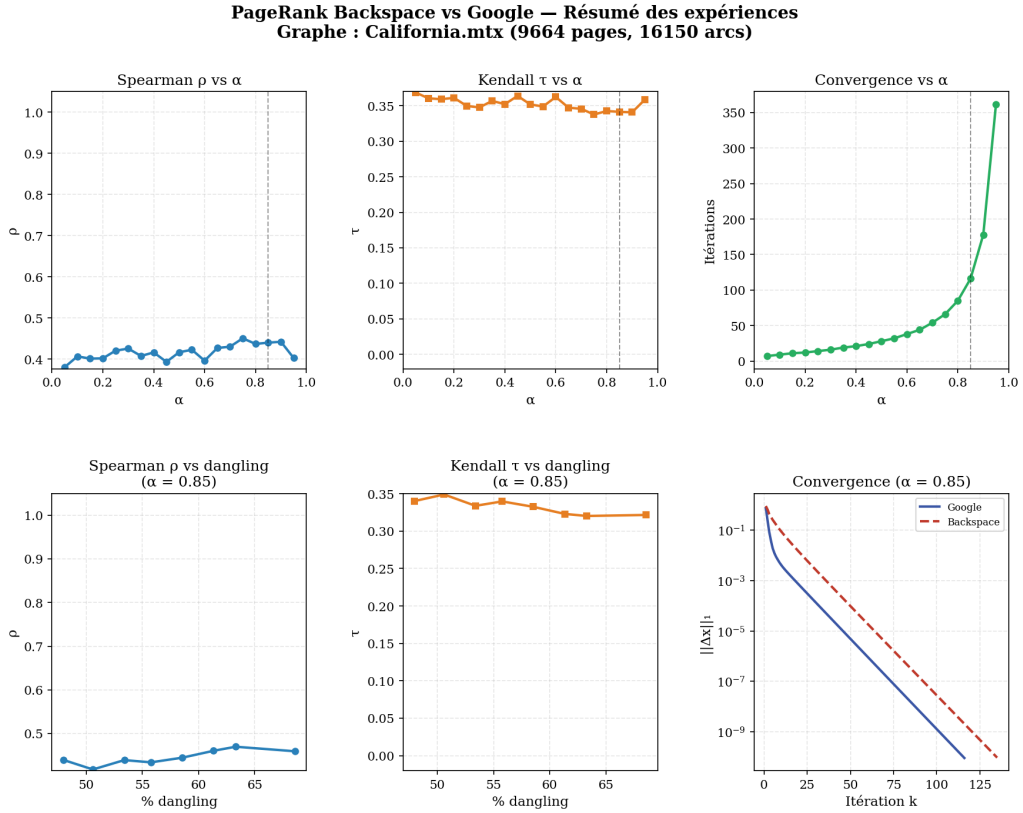


Figure 5: Dashboard résumé pour le graphe California (9 664 pages, 48 % de dangling nodes).

Interprétation. ρ oscille autour de 0.40–0.44 sans tendance monotone sur $\alpha \in [0.05, 0.95]$, et τ reste stable autour de 0.34 : à proportion intermédiaire de dangling nodes, la divergence entre les deux méthodes est présente mais indépendante de α.

6 Comparaison sur trois graphes : California, EPA et HEP-th

6.1 Présentation des trois graphes

Afin de valider la généralité de nos résultats, nous avons mené les mêmes expériences sur trois graphes de natures très différentes : un graphe web (California), un graphe environnemental (EPA) et un graphe de citations scientifiques (HEP-th).

Les trois graphes offrent une gamme représentative : de 10 % à 70 % de dangling nodes, et des densités très différentes (HEP-th est environ 13 fois plus dense que California).

6.2 Comportement de ρ et τ en fonction de α

Interprétation.

Graphe	Pages (N)	Arcs	Dangling nodes	N_{trans}	ρ ($\alpha = 0.85$)	τ ($\alpha = 0.85$)
California	9 664	16 150	4 637 (48%)	13 103	0.440	0.341
EPA	4 772	8 965	3 350 (70%)	7 671	0.515	0.303
HEP-th	27 770	352 807	2 750 (10%)	49 426	0.695	0.238

Table 4: Caractéristiques et métriques des trois graphes ($\alpha = 0.85$). N_{trans} est la taille du graphe étendu après transformation Backspace.

Les trois graphes révèlent un gradient clair piloté par la proportion de dangling nodes :

- **HEP-th (10 % de dangling nodes)** : ρ élevé (≈ 0.63–0.70) et τ faible (≈ 0.23–0.25). Les deux méthodes sont très proches ; ρ croît avec α jusqu'à 0.70, puis redescend légèrement — peu de dangling nodes signifie peu de différences à amplifier.
- **California (48 %)** : ρ oscille autour de 0.40–0.44 sans tendance claire. La divergence est présente mais stable sur tout le spectre de α.
- **EPA (70 %)** : ρ décroît régulièrement de 0.63 à 0.51 et τ croît de 0.24 à 0.31 quand α augmente. La tendance théorique ($\alpha \to 1 \Rightarrow$ plus de divergence) est ici la plus lisible, portée par la masse de dangling nodes.

Conclusion : la proportion de dangling nodes détermine à la fois l'amplitude de la divergence entre les deux méthodes et la visibilité de son évolution avec α.

6.3 Impact de la destruction de liens

- **California et EPA** : détruire des liens fait *augmenter* ρ et *diminuer* τ — les rankings semblent paradoxalement se rapprocher. Explication : les pages isolées obtiennent des scores quasi-nuls dans les deux méthodes et se classent similairement en bas du ranking, masquant les vraies divergences.
- **HEP-th** : le même paradoxe s'observe jusqu'à 25 % de destruction (ρ monte de 0.695 à 0.758), puis ρ redescend au-delà de 30 %. Cela s'explique par la très faible proportion initiale de dangling nodes : à fort taux de destruction, le graphe bascule dans un régime similaire à California et EPA, et la divergence réelle reprend le dessus.
- **Limite de calcul** : pour California et EPA, l'expérience s'arrête à 40 % de destruction (saturation des tableaux statiques et instabilité de convergence). HEP-th, malgré sa grande taille, peut aller jusqu'à 50 % car son faible taux initial de dangling nodes limite l'explosion du graphe étendu.

6.4 Comportement universel de la convergence

Les trois graphes convergent en un nombre d'itérations quasi-identique à structure fixée : 109–116 itérations pour $\alpha = 0.85$ et 332–362 pour $\alpha = 0.95$, quelle que soit la taille ou la densité du graphe. Cela confirme que le taux de convergence de l'itération de puissance est α, indépendamment de la structure du graphe — résultat théorique désormais vérifié sur trois jeux de données distincts.

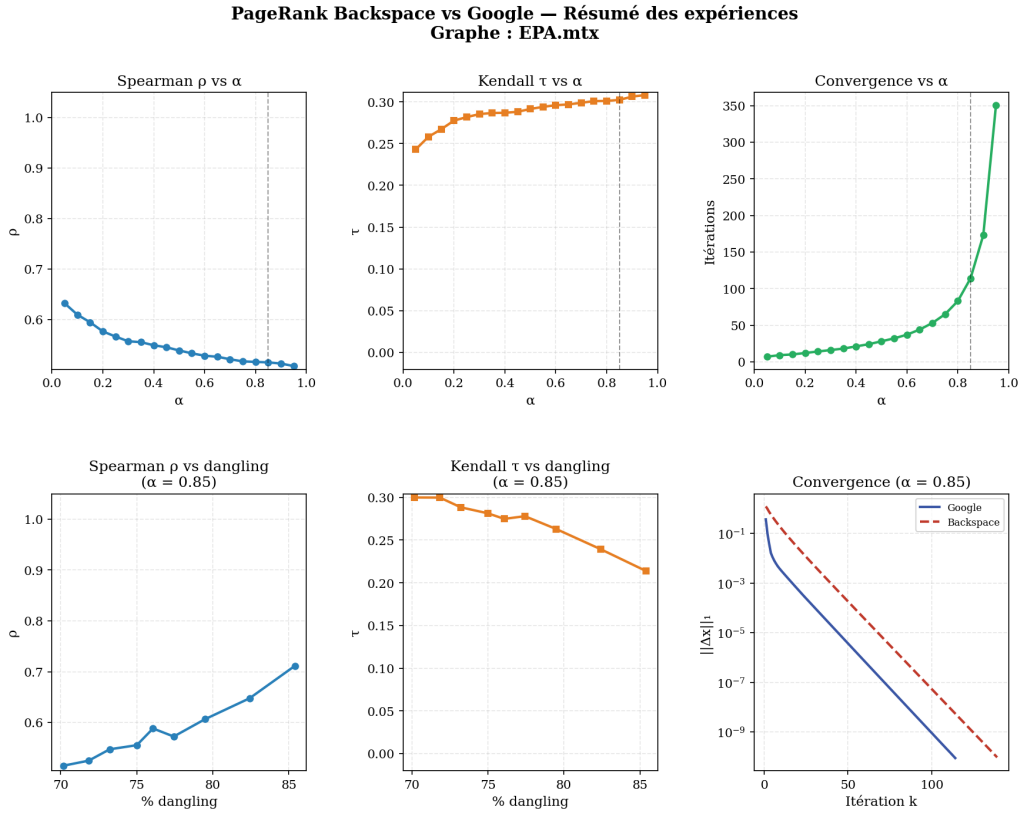


Figure 6: Dashboard résumé pour le graphe EPA (4 772 pages, 70 % de dangling nodes). ρ décroît régulièrement de 0.63 à 0.51 et τ croît de 0.24 à 0.31 quand α augmente : la forte proportion de dangling nodes rend la divergence entre les deux méthodes clairement monotone.

7 Analyse et discussion

7.1 Complexité computationnelle

Méthode	Taille	Coût par itération	Nb itérations
Google	N	$O(\mathrm{nnz}(H))$	$O\left(\frac{\log \varepsilon}{\log \alpha}\right)$
Backspace	$N_{\text{trans}} \geq N$	$O(\mathrm{nnz}(H_{\text{ext}}))$	idem

Table 5: Complexité des deux méthodes

Le surcoût de Backspace est proportionnel au nombre de clones créés, soit $\sum_{P \in D} d^-(P)$ nœuds et arcs supplémentaires. Dans le pire cas (beaucoup de dangling nodes très connectés), ce surcoût peut être significatif, mais reste linéaire en la taille du graphe.

7.2 Pertinence du modèle Backspace

L'approche Backspace repose sur une hypothèse comportementale : l'utilisateur revient exactement à la page précédente.

- C'est une modélisation plus réaliste que la redistribution uniforme de Google pour les pages vraiment « sans issue » (pages d'erreur, PDF sans hyperliens).

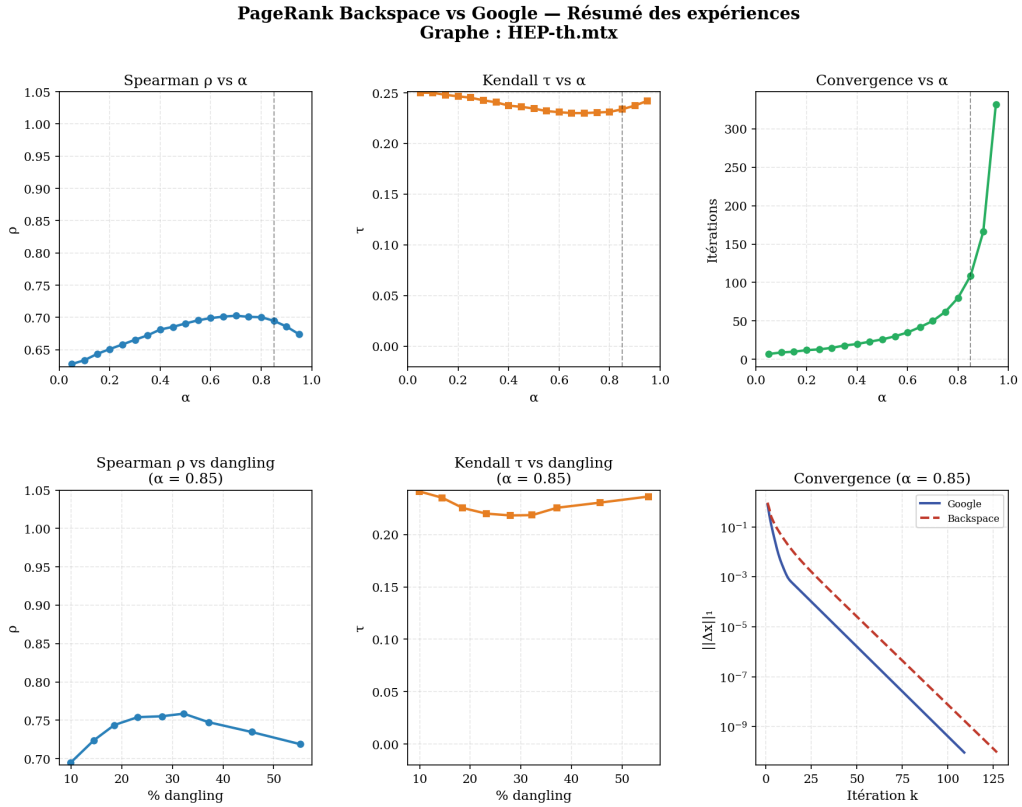


Figure 7: Dashboard résumé pour le graphe HEP-th (27 770 pages, 10 % de dangling nodes). ρ croît de 0.63 à 0.70 puis redescend légèrement, τ décroît de 0.25 à 0.23 : avec peu de dangling nodes, les deux méthodes restent proches et la tendance s'inverse par rapport à EPA.

- La duplication des nœuds augmente la taille du graphe, mais préserve la structure de Markov : la matrice d'itération reste stochastique et la convergence est garantie par le théorème de Perron-Frobenius.

7.3 Cas du parent lui-même dangling

Lors de la transformation Backspace, un prédécesseur X de P peut lui-même être dangling ($d^+(X) = 0$). Dans ce cas, `correspondance[X]` vaut -1 (initialisation de sécurité) et le lien est ignoré avec un avertissement. Ce cas est correctement détecté et traité sans crash ni comportement indéfini.

8 Conclusion

Nous avons implémenté et comparé en C deux variantes de PageRank pour traiter les dangling nodes : l'approche Google (redistribution uniforme) et l'approche Backspace (duplication des dangling nodes et retour vers le prédécesseur).

Les expériences menées sur trois graphes de natures différentes (California, EPA et HEP-th) montrent que :

1. Les deux approches produisent des rankings significativement différents ($\rho \approx 0.42$–0.47, $\tau \approx 0.32$–0.37), avec un écart qui dépend de la proportion de dangling nodes

et de α.

2. L'effet de α est directement proportionnel à la proportion de dangling nodes : clairement monotone sur EPA (70 %), stable sur California (48 %), et inversé sur HEP-th (10 %) où les deux méthodes restent très proches.

3. La convergence dépend uniquement de α, indépendamment de la structure du graphe : résultat confirmé sur les trois jeux de données.

4. L'approche Backspace favorise les prédécesseurs directs des dangling nodes, ce qui reflète mieux le comportement réel de navigation.

5. Le paradoxe observé (destruction de liens $\Rightarrow$ rankings plus similaires) est un phénomène structural reproduit sur les trois graphes, expliqué par l'isolement des pages détruites ; HEP-th montre en outre qu'au-delà d'un seuil la tendance peut s'inverser quand la proportion de dangling nodes était initialement faible.

6. Le calcul au-delà de 40 % de destruction est impossible sur California en raison de l'explosion de la taille du graphe étendu et de l'instabilité de la convergence ; ce seuil est inhérent à la complexité de la transformation Backspace avec des tableaux statiques.

7. Le cas dégénéré d'un prédécesseur lui-même dangling est correctement détecté et géré sans crash ni comportement indéfini.

References

[1] V. Batagelj, A. Mrvar, *Pajek/California — Pajek network: Kleinberg's web search of "California"*, UF Sparse Matrix Collection, 2006.

[2] T. Davis, *Pajek/EPA — EPA web graph*, UF Sparse Matrix Collection, 2006.

[3] J. Leskovec, J. Kleinberg, C. Faloutsos, *Graph Evolution:*, Graphe HEP-th issu de la Stanford Large Network Dataset Collection (SNAP).